```
  def create
    @favorite = Favorite.create(
      user: current_user,
      concert_id: params[:concert_id]
    )
    respond_to do |format|
      format.turbo_stream
    end
  end

  def destroy
    @favorite = Favorite.find(params[:id])
    @favorite.destroy
    respond_to do |format|
      format.turbo_stream
    end
  end

  private def favorite_params
    params.require(:concert_id)
  end
end
```

Both the `create` and `destroy` methods have a similar structure. In the first part, we do the actual creating or destroying of the favorite record, then we use the standard Rails `respond_to` to direct `turbo_stream` responses to the default Rails behavior, which is to look for `create.turbo_stream.erb` or `destroy.turbo_stream.erb`. Strictly speaking, since this is all default behavior, you could leave out the `respond_to` block entirely as the controller would by default search for the same `<action>.<format>.erb` file anyway; whether you do so depends on whether there are other formats you want to deal with. This structure makes it easy to do something different with a regular `html` request versus a `turbo_stream` request.

The ERB files use a new Rails helper to create Turbo Streams. Let's look at the `create` file first:

chapter_02/04/app/views/favorites/create.turbo_stream.erb

```
<%= turbo_stream.append("favorite-concerts-list", @favorite) %>
<%= turbo_stream.replace(dom_id(@favorite.concert), @favorite.concert) %>
```